HACKATHON PROBLEM STATEMENT PS-01

On-Device App Risk Auditor

Find suspicious Android apps through their permissions, special access and an explainable risk score

TRACK	DIFFICULTY	DURATION	REQUIRED OUTPUT
Mobile Security	Medium	6 hours	Working APK

01 Problem Statement

> Design and build an Android application, or a reusable module within one, that scans all user-installed apps on the device, extracts the permissions and special access each app declares and has been granted, organizes them into meaningful categories, and assigns each app a risk level with human-readable reasons.

The tool must let a non-technical user answer three questions:

Q1
Which of my apps can do the most harm?

Q2
Why is each one flagged?

Q3
What can I do about it right now?

02 Permission Categories

Teams may add categories, but every submission must cover these nine. Special access carries the highest weight because it is granted through separate settings screens and is what most banking trojans abuse.

#	CATEGORY	EXAMPLE PERMISSIONS OR ACCESS	WHY IT MATTERS
01	Special access HIGHEST WEIGHT	Accessibility service, screen overlay (SYSTEM_ALERT_WINDOW), device admin, notification listener, REQUEST_INSTALL_PACKAGES, PACKAGE_USAGE_STATS	Screen reading, fake login overlays, uninstall blocking, OTP capture from notifications, installing further apps
02	SMS	READ_SMS, RECEIVE_SMS, SEND_SMS	OTP theft, premium SMS fraud
03	Contacts and call logs	READ_CONTACTS, READ_CALL_LOG	Harvesting the social graph for scams and extortion
04	Camera and microphone	CAMERA, RECORD_AUDIO	Covert recording
05	Location	ACCESS_FINE_LOCATION, ACCESS_BACKGROUND_LOCATION	Tracking and stalking
06	Phone and identity	READ_PHONE_STATE, READ_PHONE_NUMBERS, CALL_PHONE	Device fingerprinting, call fraud
07	Storage and media	READ_MEDIA_IMAGES, MANAGE_EXTERNAL_STORAGE	Photo and document theft

#	CATEGORY	EXAMPLE PERMISSIONS OR ACCESS	WHY IT MATTERS
08	Background and persistence	RECEIVE_BOOT_COMPLETED, FOREGROUND_SERVICE, REQUEST_IGNORE_BATTERY_OPTIMIZATIONS	Survives reboots and keeps running unseen
09	Network	INTERNET, ACCESS_NETWORK_STATE	The exfiltration channel; low risk alone, high in combination

03 Functional Requirements

MUST HAVE The minimum for a passing demo

01	App inventory	Name, icon, package name, version, install date, installer source and target SDK for every user-installed app, with a toggle to include system apps.
02	Permission extraction	Declared versus granted for each app, with each permission's protection level (normal, dangerous, signature, special).
03	Categorization	Group permissions into the nine categories listed in Section 2, shown per app and as a device-wide summary.
04	Special access detection	Enabled accessibility services, active device admins, enabled notification listeners, and declared overlay and install-packages access.
05	Risk score	Score from 0 to 100 with four bands (Low, Medium, High, Critical) and at least three plain-language reasons for every High or Critical app.
06	App detail screen	A "Why flagged" panel, a button to the app's system settings page, and an uninstall button.

SHOULD HAVE Expected from strong teams

07	Combination rules	For example: SMS read plus internet plus boot persistence from a sideloaded source; accessibility plus overlay; device admin on an app that is not a security or MDM tool.
08	Context mismatch	Flag apps whose label or Play category does not fit their access, such as a torch, calculator or wallpaper app holding SMS, contacts or call log access.
09	Install monitoring	Rescan when an app is installed or updated and notify the user if it scores High or Critical.
10	Permission diff	Alert when an update adds new sensitive permissions.

STRETCH	For teams that finish early	
11	Hash lookup	Compute the SHA-256 of each base APK and check it against a threat intelligence service. Send hashes only, never the APK.
12	Static manifest checks	Inspect the APK file for exported components, cleartext traffic allowed and the debuggable flag.
13	On-device ML model	Trained before the event on a public permission dataset and shown beside the rule score. The rule score stays the source of explanations.
14	Shareable PDF report	Export the full device scan as a PDF report.

04 Constraints

On-device only	All scanning runs on the device, with no root and no ADB needed by the end user.
Privacy	No user data leaves the device, except optional hash lookups after explicit consent.
Offline first	Core features work offline.
Package visibility	QUERY_ALL_PACKAGES may be declared for the hackathon, even though Play Store policy restricts it for published apps.
No live malware	No real malware samples may be used or distributed.

05 Deliverables

Every team submits all four items. Judging uses only the organizer test kit and judging device.

#	DELIVERABLE	DETAILS
01	Working APK	Installed and running on the judging device or emulator by the end of hour 5.
02	Scoring rubric	One page listing the risk weights and combination rules.
03	Source code	Link to the source repository.
04	Live demo	A 3-minute demo run on the organizer test kit, not on apps the team chose.